



Faculty of Electrical and Computer Engineering

Workshop Acceptance Testing

Presented by: Team 3

Members:

Eimmy Ochoa Morán - eimvocho@espol.edu.ec

José Toapanta Dominguez - jatoapan@espol.edu.ec

Sebastian Holguin Vargas - sebhvarg@espol.edu.ec

Issac Maza Punine - issamaza@espol.edu.ec

Leonardo Zambrano Toala - ltzambra@espol.edu.ec

Teacher:

Dr. Carlos Mera Gómez

Guayaquil – Ecuador

2026



Content

Introduction.....	3
Development.....	3
Tool Configuration & Code Creation.....	3
Test Run Evidence.....	9
First feature.....	9
Second Feature.....	9
Third Feature.....	10
Fourth Feature.....	10
Fifth Feature.....	11
Iterative Process.....	11
Conclusions and Recommendations.....	11
Repository.....	12
References.....	12



Introduction

In the world of software development, creating a program that simply runs without crashing is not enough; it must work exactly as the client expects. This workshop aims to demonstrate how we can build a reliable application by putting the user's needs first.

To achieve this, our team adopted a collaborative approach called Behavior Driven Development (BDD). Instead of jumping straight into complex programming, BDD encourages us to first discuss and write down how the system should behave in everyday situations. By writing our test scenarios in plain English, describing actions like adding a new product or searching for one that doesn't exist. Once these expectations are set, we use specialized tools to automatically check our Python code against these scenarios, ensuring that our final product is highly functional and perfectly aligned with the original requirements.

Development

Tool Configuration & Code Creation

```
(venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\workshop-acceptance-testing-T3> pip install behave
Requirement already satisfied: behave in c:\users\leoza\desktop\espol\septimosemestre\ingsw2\workshop-acceptance-testing-T3\venv\lib\site-packages (from behave) (10.0.0)
Requirement already satisfied: cucumber-tag-expressions>=4.1.0 in c:\users\leoza\desktop\espol\septimosemestre\ingsw2\workshop-acceptance-testing-T3\venv\lib\site-packages (from behave) (10.0.0)
Requirement already satisfied: cucumber-expressions>=17.1.0 in c:\users\leoza\desktop\espol\septimosemestre\ingsw2\workshop-acceptance-testing-T3\venv\lib\site-packages (from behave) (20.0.0)
Requirement already satisfied: parse>=1.18.0 in c:\users\leoza\desktop\espol\septimosemestre\ingsw2\workshop-acceptance-testing-T3\venv\lib\site-packages (from behave) (1.22.1)
Requirement already satisfied: parse-type>=0.6.0 in c:\users\leoza\desktop\espol\septimosemestre\ingsw2\workshop-acceptance-testing-T3\venv\lib\site-packages (from behave) (0.6.6)
Requirement already satisfied: six>=1.15.0 in c:\users\leoza\desktop\espol\septimosemestre\ingsw2\workshop-acceptance-testing-T3\venv\lib\site-packages (from behave) (1.17.0)
Requirement already satisfied: colorama>=0.3.7 in c:\users\leoza\desktop\espol\septimosemestre\ingsw2\workshop-acceptance-testing-T3\venv\lib\site-packages (from behave) (0.4.6)

[notice] A new release of pip available: 22.3.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\workshop-acceptance-testing-T3> pip show behave
Name: behave
Version: 1.3.3
Summary: behave is behaviour-driven development, Python style
Home-page:
Author: Benno Rice, Richard Jones
Author-email: Jens Engel <jenisys@noreply.github.com>
License:
Location: C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\workshop-acceptance-testing-T3\venv\Lib\site-packages
Requires: colorama, cucumber-expressions, cucumber-tag-expressions, parse, parse-type, six
Required-by:
(venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\workshop-acceptance-testing-T3> █
```

Before testing, we set up an isolated virtual environment (venv) and installed the "Behave" framework. Behave acts as the bridge that connects our plain English scenarios with our Python code to automatically verify the application's behavior. The console output below confirms that Behave and its dependencies were successfully installed and are ready to use.

inventory.py

```

1 import sys
2
3 class Product:
4     def __init__(self, name, category, price, quantity):
5         self.name = name
6         self.category = category
7         self.price = float(price)
8         self.quantity = int(quantity)
9
10    def __str__(self):
11        return f"{self.name} | Category: {self.category} | Price: ${self.price:.2f} | Quantity: {self.quantity}"
12
13 class Inventory:
14    def __init__(self):
15        self.products = {}
16
17    def update_quantity(self, name, new_quantity):
18        if name not in self.products:
19            return False, f"Product '{name}' not found."
20        self.products[name].quantity = int(new_quantity)
21        return True, f"Quantity of '{name}' updated to {new_quantity}."
22
23    def remove_product(self, name):
24        if name not in self.products:
25            return False, f"Product '{name}' not found."
26        del self.products[name]
27        return True, f"Product '{name}' removed successfully."
28
29    def add_product(self, name, category, price, quantity):
30        if name in self.products:
31            return False, f"Product '{name}' already exists."
32        self.products[name] = Product(name, category, price, quantity)
33        return True, f"Product '{name}' added successfully."
34
35    def search_product(self, name):
36        """Feature 5: Search for a specific product by name"""
37        if name not in self.products:
38            return False, f"Product '{name}' not found."
39        return True, self.products[name]
40
41    def list_products(self):
42        return list(self.products.values())

```

```

43 def main():
44     inventory = Inventory()
45     inventory.add_product("Coffee", "Beverage", 5.5, 10)
46     inventory.add_product("Sugar", "Food", 2.0, 20)
47
48
49     print("Welcome to Inventory Manager")
50     while True:
51         print("\n--- Menu ---")
52         print("1. Add product")
53         print("2. List products")
54         print("3. Update quantity")
55         print("4. Remove product")
56         print("5. Search product")
57         print("6. Exit")
58
59         choice = input("Enter choice (1-6): ")
60
61         if choice == '1':
62             name = input("Name: ")
63             category = input("Category: ")
64
65             try:
66                 price = float(input("Price: "))
67                 quantity = int(input("Quantity: "))
68                 _, msg = inventory.add_product(name, category, price, quantity)
69                 print(msg)
70             except ValueError:
71                 print("Error: Price must be a number and Quantity must be an integer.")
72
73         elif choice == '3':
74             name = input("Product Name: ")
75             try:
76                 quantity = int(input("New Quantity: "))
77                 success, msg = inventory.update_quantity(name, quantity)
78                 print(msg)
79             except ValueError:
80                 print("Error: Quantity must be an integer.")

```

```

81
82         elif choice == '4':
83             name = input("Product Name: ")
84             success, msg = inventory.remove_product(name)
85             print(msg)
86
87         elif choice == '5':
88             name = input("Product Name: ")
89             success, result = inventory.search_product(name)
90             if success:
91                 print(f"Found: {result}")
92             else:
93                 print(result)
94
95         elif choice == '2':
96             products = inventory.list_products()
97             if not products:
98                 print("Inventory is empty.")
99             else:
100                 for p in products:
101                     print(p)
102
103         elif choice == '6':
104             print("Exiting...")
105             sys.exit(0)
106         else:
107             print("Invalid choice. Please try again.")
108
109     if __name__ == "__main__":
110         main()
111

```

This file contains the core logic of the Inventory Manager application. It defines two main classes: the Product class, which models the data structure of an item (name, category, price, and quantity), and the Inventory class, which manages a dictionary of products. The Inventory class includes all the methods necessary to fulfill the business requirements: add_product, list_products, update_quantity, remove_product, and search_product. Furthermore, it implements a Command Line Interface (CLI) within the main() function to allow user interaction, input handling, and manual validation of the features, actively pre-populating the inventory with default items (such as "Coffee" and "Sugar") to facilitate immediate testing.

inventory_steps.py

```

9  # =====
10 # FEATURE: SHARED / COMMON STEPS
11 # =====
12 @given('the inventory is empty')
13 def step_impl_empty_inventory(context):
14     context.inventory = Inventory()
15     # Adding base elements so the list is never completely empty
16     context.inventory.add_product("Base Mouse", "Electronics", 15.0, 10)
17     context.inventory.add_product("Base Keyboard", "Electronics", 25.0, 5)
18
19 @given('the inventory contains products:')
20 def step_impl_given_products(context):
21     context.inventory = Inventory()
22     # Adding base elements so the list is never completely empty
23     context.inventory.add_product("Base Mouse", "Electronics", 15.0, 10)
24     context.inventory.add_product("Base Keyboard", "Electronics", 25.0, 5)
25     for row in context.table:
26         product_name = row['Product']
27         context.inventory.add_product(product_name, "Groceries", 5.0, 10)
28
29 @then('the output should be "{message}"')
30 def step_impl_check_output(context, message):
31     assert context.output == message, f'Expected "{message}" but got "{context.output}"'
32
33
34 # =====
35 # FEATURE: ADD PRODUCT
36 # =====
37 @when('the user adds a product "{name}"')
38 def step_impl_add_product(context, name):
39     context.inventory.add_product(name, category="Beverage", price=5.0, quantity=10)
40
41 @then('the inventory should contain "{name}"')
42 def step_impl_check_contains(context, name):
43     assert name in context.inventory.products, f'Product "{name}" not found in the inventory'
44

```

```

47 # FEATURE: SEARCH PRODUCT
48 # =====
49 @when('the user searches for the product "{product}"')
50 def step_impl_search_product(context, product):
51     context.success, context.output = context.inventory.search_product(product)
52
53
54 # =====
55 # FEATURE: UPDATE PRODUCT
56 # =====
57 @when('the user updates product "{name}" to quantity "{quantity}"')
58 def step_impl_update_product(context, name, quantity):
59     _, msg = context.inventory.update_quantity(name, quantity)
60     context.output = msg
61
62
63 # =====
64 # FEATURE: REMOVE PRODUCT
65 # =====
66 @when('the user removes the product "{product_name}"')
67 def step_impl_remove_product(context, product_name):
68     _, msg = context.inventory.remove_product(product_name)
69     context.output = msg
70
71 @then('the inventory should not contain "{product_name}"')
72 def step_impl_should_not_contain(context, product_name):
73     assert product_name not in context.inventory.products, f"Error: '{product_name}' was not deleted."
74
75
76 # =====
77 # FEATURE: LIST PRODUCTS
78 # =====
79 @when('the user lists all products')
80 def step_impl_list_products(context):
81     context.output = context.inventory.list_products()
82
83 @then('the output should contain:')
84 def step_impl_report_list(context):
85     product_names = [product.name for product in context.output]
86     for row in context.table:
87         assert row["Product"] in product_names

```

This file acts as the critical bridge between the plain English scenarios defined in Gherkin and the actual Python application logic. It utilizes Behave decorators (@given, @when, @then) to map each line of the .feature file to specific Python functions. Within these functions, we instantiate the Inventory class and heavily rely on the context object to safely share data and state across different steps (such as storing the context.inventory instance or capturing context.output messages for failed interactions). Additionally, this file handles advanced testing mechanics, such as parsing Gherkin data tables (context.table) to efficiently populate or validate multiple items, and employs Python assert statements to strictly verify that the application's responses perfectly match the expected outcomes.

inventory.feature

```

1  Feature: Inventory Manager System
2
3      Scenario: Add a product to an existing inventory
4      Given the inventory contains products:
5          | Product |
6          | Apple  |
7      When the user adds a product "Banana"
8      Then the inventory should contain "Banana"
9
10
11     Scenario: Search for a product that does not exist
12     Given the inventory is empty
13     When the user searches for the product "Mouse"
14     Then the output should be "Product 'Mouse' not found."
15
16     Scenario: Update a product that does not exist
17     Given the inventory is empty
18     When the user updates product "Tea" to quantity "20"
19     Then the output should be "Product 'Tea' not found."
20
21     Scenario: Remove a product from the inventory
22     Given the inventory contains products:
23         | Product |
24         | Coffee |
25         | Sugar  |
26     When the user removes the product "Coffee"
27     Then the inventory should not contain "Coffee"
28
29     Scenario: Remove a product that does not exist
30     Given the inventory is empty
31     When the user removes the product "Coffee"
32     Then the output should be "Product 'Coffee' not found."
33
34     Scenario: List all products in the inventory
35     Given the inventory contains products:
36         | Product |
37         | Coffee |
38         | Sugar  |
39     When the user lists all products
40     Then the output should contain:
41         | Product |
42         | Coffee |
43         | Sugar  |

```

This file serves as the executable specification of the system, written in plain English using the Gherkin syntax. Under a single main Feature: Inventory Manager System, it outlines six distinct test scenarios that validate both successful operations (such as adding, removing, and listing products using data tables) and expected failure cases (like attempting to update, search, or remove a product that does not exist). By using the Given-When-Then structure, these scenarios define the exact preconditions, user actions, and expected system outputs, which are then automated and evaluated using the Behave framework.

Test Run Evidence

First feature

Evidence:

```
~/Documentos/workshop-acceptance-testing-T3 main* 14s
venv > behave
USING RUNNER: behave.runner:Runner
Feature: Inventory Manager # features/inventory.feature:1

  Scenario: Add a product to the inventory # features/inventory.feature:3
    Given the inventory is empty # features/steps/inventory_steps.py:10 0.000s
    When the user adds a product "Coffee" # features/steps/inventory_steps.py:14 0.000s
    Then the inventory should contain "Coffee" # features/steps/inventory_steps.py:19 0.000s

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 0min 0.000s

~/Documentos/workshop-acceptance-testing-T3 feature-1-AddProduct*
venv > echo "Sebastian Holguin"
"Sebastian Holguin"
```

Second Feature

Evidence:

```
(venv) C:\Users\ClitControl\Documents\workshop-acceptance-testing-T3>python -m behave
USING RUNNER: behave.runner:Runner
Feature: Inventory Manager # features/inventory.feature:1

  Given the inventory contains products: # features/steps/inventory_steps.py:4 0.000s
  | Product | entory contains products: # features/steps/inventory_steps.py:4
  | Coffee |
  Then the output should contain: # features/steps/inventory_steps.py:21 0.000s
  | Product | ut should contain: # features/steps/inventory_steps.py:21
  | Coffee |
  | Sugar |
  | Sugar |

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 0min 0.000s

(venv) C:\Users\ClitControl\Documents\workshop-acceptance-testing-T3> echo "Issac Alexander Maza Punine"
"Issac Alexander Maza Punine"
```

Third Feature

Evidence:

```

MINGW64~/Users/joset/Downloads/CIAP/workshop-acceptance-testing-T3
(venv)
joset@MSI MINGW64 ~/Downloads/CIAP/workshop-acceptance-testing-T3 (feature-3-UpdateProduct)
$ behave
USING RUNNER: behave.runner:Runner
Feature: Inventory Manager # features/inventory.feature:1

  Scenario: Update a product that does not exist # features/inventory.feature:3
    Given the inventory is empty # features/steps/inventory_steps.py:
    When the user updates product "Tea" to quantity "20" # features/steps/inventory_steps.py:
    Then the output should be "Product 'Tea' not found." # features/steps/inventory_steps.py:

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 0min 0.000s
(venv)
joset@MSI MINGW64 ~/Downloads/CIAP/workshop-acceptance-testing-T3 (feature-3-UpdateProduct)
$ |

```

Fourth Feature

Evidence:

```

(venv) PS C:\Users\Eimmy Ochoa\Documents\GitHub\workshop-acceptance-testing-T3> behave
USING RUNNER: behave.runner:Runner
Feature: Inventory Manager System # features/inventory.feature:1

  Then the inventory should contain "Coffee" # features/steps/inventory_steps.py:19 0.000s
  Then the inventory should contain "Coffee" # features/steps/inventory_steps.py:19
  Given the inventory contains products: # features/steps/inventory_steps.py:27 0.000s
    | Product |entory contains products: # features/steps/inventory_steps.py:27
    | Coffee |
  Then the inventory should not contain "Coffee" # features/steps/inventory_steps.py:46 0.000s
  Then the inventory should not contain "Coffee" # features/steps/inventory_steps.py:46
  Then the output should be "Product 'Coffee' not found." # features/steps/inventory_steps.py:51 0.000s
  Then the output should be "Product 'Coffee' not found." # features/steps/inventory_steps.py:51

1 feature passed, 0 failed, 0 skipped
3 scenarios passed, 0 failed, 0 skipped
9 steps passed, 0 failed, 0 skipped
Took 0min 0.006s
(venv) PS C:\Users\Eimmy Ochoa\Documents\GitHub\workshop-acceptance-testing-T3>

```

Fifth Feature

Evidence:

```
(venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\workshop-acceptance-testing-T3> behave
USING RUNNER: behave.runner:Runner
Feature: Inventory Manager # features/inventory.feature:1

    Then the inventory should contain "Coffee" # features/steps/inventory_steps.py:19 0.000s
    Then the inventory should contain "Coffee" # features/steps/inventory_steps.py:19
    Then the output should be "Product 'Mouse' not found." # features/steps/inventory_steps.py:29 0.000s
    Then the output should be "Product 'Mouse' not found." # features/steps/inventory_steps.py:29
1 feature passed, 0 failed, 0 skipped
2 scenarios passed, 0 failed, 0 skipped
6 steps passed, 0 failed, 0 skipped
Took 0min 0.008s
(venv) PS C:\Users\leoza\Desktop\ESPOL\SeptimoSemestre\IngSW2\workshop-acceptance-testing-T3>
```

Iterative Process

Following the project configuration and the creation of the step definitions, the team executed the acceptance tests using the Behave framework. We carefully reviewed our previous results and verified the interaction between the inventory.py logic and the inventory_steps.py assertions. During this iterative review process, no errors or failures were found. All five developed features and scenarios passed successfully on the first test run, requiring no further code corrections.

Conclusions and Recommendations

Implementing Behavior Driven Development (BDD) alongside the Gherkin syntax has proven to be a highly effective strategy for clarifying project goals before writing the actual code. By defining the software requirements, our team gained a much deeper understanding of the expected system behaviors and edge cases. Automating these acceptance tests with the Python Behave framework provided a reliable safety net for our development process. Reducing the risk of introducing regressions during future code modifications.

For future projects utilizing this testing framework, we strongly recommend paying close attention to the strict formatting of the .feature files. The Gherkin parser is highly sensitive to

syntax and indentation errors, meaning that even a misplaced space or an unsupported declaration can cause the entire test suite to fail.

Additionally, developers should always ensure they isolate their project workspaces by using a virtual environment (venv). This practice is crucial to guarantee that the automated tests can be reliably reproduced across different machines and operating systems without version mismatches.

Repository

<https://github.com/Sebhvarg/workshop-acceptance-testing-T3>

References

- Behave Documentation. (n.d.). *Welcome to behave!* Retrieved from <https://behave.readthedocs.io/>
- Cucumber. (n.d.). *Gherkin Reference*. Retrieved from <https://cucumber.io/docs/gherkin/reference/>